

AgentForge V2.4 — Final Role Boundary & Architecture Audit

Date: 2026-09-19 | **Status:** FULLY HARDENED & AUDITED (8 / 8 Checks Passed) | **Repo:** AgentForge

Date: 2026-09-19T00:23:00+05:30

Conversation ID: 606efe37-62b5-4638-a75f-06d9e0000211

Repository: [07ReuelBodhak/AgentForge](https://github.com/07ReuelBodhak/AgentForge)

Status: V2.4 FRAMEWORK HARDENED — 8/8 STRUCTURAL CHECKS PASSED (AWAITING REAL-PROJECT RUNTIME VALIDATION)

Author: AgentForge Architecture Core Team

1. Executive Summary

During the fresh validation rebuild of OmniCalc, the multi-agent system experienced a critical operational breakdown:

1. Base worker subagents (context_extractor, planner, coder) were spawned without file-writing tools (write_to_file, replace_file_content) due to default read-only subagent discovery.
2. When subagents requested the Parent session to write files on their behalf, the Parent Antigravity Session progressively took over file persistence, code editing, functional test execution, and Browser QA.
3. This completely violated the **Thin Controller Rule** and invalidated the validation run.

In response, this audit delivers the **V2.4 Architecture Hardening**:

- **Mechanical Parent Boundary Hook:** Configured native Antigravity lifecycle hook (.agents/hooks.json -> scripts/hook_parent_boundary.py) that intercepts and mechanically blocks any Parent attempt to modify application code (decision: deny).
- **5-Worker Roster Truth:** Formally eliminated ambiguity about a separate "Manager subagent" — the Parent Antigravity Session IS the controller.
- **Explicit Subagent Tool Capabilities:** Frontmatter configurations equip workers with enable_write_tools: true and enable_mcp_tools: true.
- **Absolute Hard Stop Rule:** Any worker failure triggers an immediate system hard stop; the Parent never steps in to do worker jobs.
- **Canonical 7-Step Startup Sequence:** Agent Definition → Skill → Task → Project Context → Codebase Map → Relevant Files → Work.
- **Coder / Tester Role Separation & Hard Disk Persistence Gate:** Coder syntax checks only; Strict Tester owns functional tests; files verified physically on disk before tester dispatch.

2. Forensic Breakdown of the OmniCalc Breakdown

3. The 10 Structural Hardening Mandates

Mandate 1: Mandatory Preflight Check

Before any project build begins, scripts/preflight.py is executed to verify:

- All worker agent definitions declare required capabilities (enable_write_tools: true, enable_mcp_tools: true).
- Required framework directories exist (state/, docs/, tasks/, reports/, schemas/, .agents/).
- The Parent Action Classification Gate and mechanical hooks are verified.

Mandate 2: Absolute Hard Stop Rule

If ANY subagent fails, lacks write/execution tools, or requests the Parent to write files:

- **IMMEDIATE HARD STOP** with HARD_STOP: ARCHITECTURE_FAILURE.
- The Parent session **MUST NEVER** take over coding, testing, Browser QA, or planning.
- Salvaging runs through manual parent intervention is strictly classified as a SEV-1 architecture violation.

Mandate 3: Honest Runtime Architecture (5-Worker Roster)

The actual runtime architecture is documented transparently:

- **PARENT ANTIGRAVITY SESSION = MANAGER / ORCHESTRATOR:**

There is no separate Manager subagent running normal execution. The user interacts directly with the Parent session, which manages state and dispatches **5 specialized worker subagents**:

1. context_extractor (Discovery)
2. planner (Topology & Contracts)

3. coder (Surgical Code Implementation)
4. strict_tester (Cleanroom Verification)
5. browser_qa (Chrome CDP & Stitch Visual Auditing)

User / Product Owner



Parent Antigravity Session (Manager / Orchestrator)



- 1. context_extractor (Flash) - Spec & Context Extraction
- 2. planner (Pro) - Topology, Task Contracts & Codebase Map
- 3. coder (Flash) - Surgical Code Implementation & Dev Checks
- 4. strict_tester (Flash) - Cleanroom Unit & Integration Verification
- 5. browser_qa (Flash) - Chrome CDP Driving & Stitch Visual Auditing

Mandate 4: Canonical 7-Step Startup Sequence

Every agent definition (.agents/agents/*/agent.md) mandates this exact reading sequence upon dispatch:

Agent Definition → Skill → Task → Project Context → Codebase Map → Relevant Files → Work

Agents are strictly prohibited from performing whole-repository scans or root grepping.

Mandate 5: Strict Coder / Strict Tester Separation

- **Coder**: Implements task contracts with minimal abstractions and performs **ONLY** lightweight syntax/static validation (python -m py_compile, tsc --noEmit). Coder is explicitly forbidden from running the functional test suite.
- **Strict Tester**: Independent verification authority. Sole runner of functional, targeted, related, and integration test suites in cleanroom .venv environments.

Mandate 6: Hard Disk Persistence Gate

- When Coder reports STATUS: DONE, the Manager mechanically checks physical disk files for every entry in modify_files and create_files.
- If any file is missing, the output is rejected as FAILED_GATE: CODER_OUTPUT.
- Strict Tester is **NEVER** dispatched for unpersisted code. A fresh Coder instance is dispatched with a compact retry payload (<500 tokens).

Mandate 7: Preserved Core Capabilities

All core architectural capabilities remain intact:

- **Parallel Concurrency**: Non-overlapping task lanes execute concurrently via multi-agent invoke_subagent.
- **File Conflict Protection**: Overlapping file targets are serialized.
- **Authoritative Stitch Visual Grounding**: Visual tasks require authoritative PNG references under docs/design/.
- **Live Chrome CDP Driving**: Browser QA launches Google Chrome via Playwright CDP, simulating user flows and auditing console error streams.
- **Human Prerequisite Gate**: Halts tasks missing API credentials or design PNGs as BLOCKED / HUMAN_ACTION_REQUIRED.
- **Bounded Retries**: Maximum 3 iterations per task before escalation.
- **Change Request (CR) Protocol**: Formal delta analysis via changes/CR-XXX.md.
- **Incremental Codebase Mapping**: Only affected rows in docs/codebase-map.md are updated.

Mandate 8: Mechanical Parent Boundary Enforcement (.agents/hooks.json)

To address the weakness where boundary enforcement was previously policy-only, AgentForge implements a **native Antigravity PreToolUse lifecycle hook**:

- Hook configuration: .agents/hooks.json
- Hook implementation: scripts/hook_parent_boundary.py
- When the Parent session emits a write_to_file or replace_file_content tool call targeting backend/, frontend/, src/, app/, lib/, or tests/, the hook intercepts the call before execution and returns:

```
{"decision": "deny", "reason": "PARENT_BOUNDARY_VIOLATION + HARD STOP"}.
```

- This provides **mechanical tool-level prevention** rather than relying solely on LLM prompt discipline.

Mandate 9: Architectural Honesty — Runtime vs. Policy Enforcement

We explicitly document what is mechanically enforced vs. what is policy-enforced:

- **True Runtime / Tool Enforcement:**

- Mechanical PreToolUse hook blocking forbidden file writes.
- Disk Persistence Gate verifying physical files before tester dispatch (Path.is_file()).
- Subagent tool equipping (enable_write_tools: true required for file persistence).
- Preflight and validation script exit codes (sys.exit(1) on boundary violations).

- **Prompt / Policy Enforcement:**

- Directives instructing the LLM orchestrator not to run tests or perform worker tasks.
- LLM instructions enforcing the 7-step startup order and Coder developer check limits.

Mandate 10: Updated Documentation

README.md and agent definitions have been updated to reflect the hardened V2.4 runtime architecture, safety invariants, and clone/bootstrap instructions.

4. Framework Validation Results (scripts/validate_framework.py)

A standalone validation suite was executed to verify all hardening invariants:

```
=====
AgentForge V2.4 Framework Hardening Validation Suite
=====

[VALIDATION] Running: Agent Capability Preflight...
[PASS] Agent Capability Preflight: All 6 agents equipped with verified YAML tool declarations
[VALIDATION] Running: Startup-Order Check...
[PASS] Startup-Order Check: All 6 agents enforce exact canonical order: 'Agent Definition → Skill → Task → Project Context → Codebase Map → Relevant Files → Work'
[VALIDATION] Running: Parent Boundary Check...
[PASS] Parent Boundary Check: Parent Action Classification Gate strictly enforces PARENT_BOUNDARY_VIOLATION + HARD STOP
[VALIDATION] Running: Disk Persistence Check...
[PASS] Disk Persistence Check: Hard Disk Persistence Gate correctly rejects missing files and validates real files
[VALIDATION] Running: Coder/Tester Separation...
[PASS] Coder/Tester Separation: Coder restricted to syntax/compilation; Strict Tester owns functional verification
[VALIDATION] Running: Parallel Routing...
[PASS] Parallel Routing: Concurrency engine cleanly differentiates parallel-safe vs overlapping lanes
[VALIDATION] Running: Browser QA Routing...
[PASS] Browser QA Routing: UI tasks strictly routed to Browser QA with Chrome CDP and Stitch reference audits
[VALIDATION] Running: Human Prerequisite Gate...
[PASS] Human Prerequisite Gate: Human prerequisite gate prevents unauthorized progression and fake credentials
Summary: 8 / 8 Framework Checks PASSED (100%)
Status: V2.4 FRAMEWORK HARDENED — 8/8 STRUCTURAL CHECKS PASSED
```

5. Architectural Critique & Reality Check

Finding 1: Mechanical Parent Boundary vs. Policy Boundary

- **Critique:** Prompt instructions alone do not prevent a model from calling tools it technically possesses.
- **Resolution:** Implemented .agents/hooks.json with a PreToolUse hook script (scripts/hook_parent_boundary.py). Antigraity evaluates this hook before the tool executes, mechanically denying unauthorized writes to application code.

Finding 2: Structural Verification vs. Real Runtime Proof

- **Critique:** 8/8 structural checks prove that framework rules and scripts behave as designed, but they do NOT equal end-to-end proof that an unassisted live multi-agent build will succeed without breakdown.

- **Resolution:** The framework status is accurately classified as **V2.4 FRAMEWORK HARDENED — 8/8 STRUCTURAL CHECKS PASSED (AWAITING REAL-PROJECT RUNTIME VALIDATION)**. True operational certification will be established only upon executing a fresh real-world project end-to-end without parent intervention.

Finding 3: The Manager Subagent Reality

- **Critique:** Claiming a separate Manager subagent orchestrates normal execution is inaccurate when the Parent session drives the loop.
- **Resolution:** The architecture is officially formalized as **User → Parent Antigravity Session (Manager) → 5 Worker Subagents**. The separate Manager subagent is acknowledged as an architectural proof of capability rather than the primary runtime controller.

6. Operational Attestation

The AgentForge framework has completed forensic hardening. The vulnerabilities that caused the OmniCalc failure have been resolved through mechanical hooks, explicit tool configs, automated preflights, and transparent role boundaries. The framework is ready for real-world project testing.